

Ein Programm schreiben – Teil 2

Entwicklungszyklen, Schleifen und Cast

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

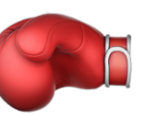
Prof. Dr. Martin Burger { 27. November 2025 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan  überlebt den ersten Kontakt  mit der Realität .

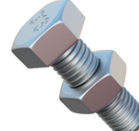
Frei nach Helmuth Karl
Bernhard von Moltke

Ein Programm schreiben

 Plan für diese Woche

 Abstrakter Entwurf

 Eine Klasse entwickeln

 Testcode

 Hauptcode

 Entwicklungszyklen

 Pair Programming

 Schleifen

 Elementare Typen casten

 Spaß

📖 Ein Programm schreiben

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

👤 Welche Schritte beinhaltet TDD?

↻ Welche Arten von Schleifen gibt es? Wie können Sie kontrollieren, wann eine Schleife beendet wird?

👖 Was kann schiefgehen, wenn man eine Long-Variable zu einer short-Variable „zurechtstutzt“?

👤 Welche Fragen haben Sie zu diesem Kapitel?

5 Ein Programm schreiben

Methoden mit Superkräften



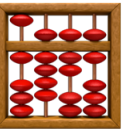
Jetzt wollen wir unsere Methoden mal etwas aufmöbeln. Wir haben uns mit Variablen beschäftigt, mit ein paar Objekten herumgespielt und ein bisschen Code geschrieben. Aber wir waren schwach. Wir brauchen Werkzeuge. Wie **Operatoren**. Und wir brauchen mehr Operatoren, um interessante Dinge zu tun, zum Beispiel *bell**en*. Und **Schleifen**. Wir brauchen Schleifen – mehr als diese mickrigen *while*-Schleifen. Eigentlich brauchen wir **for**-Schleifen. Sie könnten nützlich sein, um **Zufallszahlen zu erzeugen**. Am besten lernen wir das gleich mit. Und warum lernen wir das alles nicht, indem wir sofort etwas Reales *bauen*? Dann sehen wir, wie es ist, ein Programm von Grund auf neu zu coden (und zu testen). **Vielleicht ein Spiel** wie Schiffe versenken. So eine schwere Aufgabe müssen wir erst einmal gestemmt bekommen. Deshalb lassen wir uns auch *zwei* Kapitel Zeit, um die Aufgabe abzuschließen. In diesem Kapitel schreiben wir eine einfache Version. In Kapitel 6 bauen wir dann die De-luxe-Fassung – *mithilfe der Java-Bibliothek*.



Entwicklungszyklen

Grundidee

Entwicklungszyklen

 Umrechnung von Temperaturangaben zwischen Celsius und Fahrenheit.

 Benutzer:in gibt die Ausgangstemperatur ein.

 Benutzer:in bekommt die umgerechnete Temperatur angezeigt.



Write a list of
test cases

Pick one
test

Write the
test

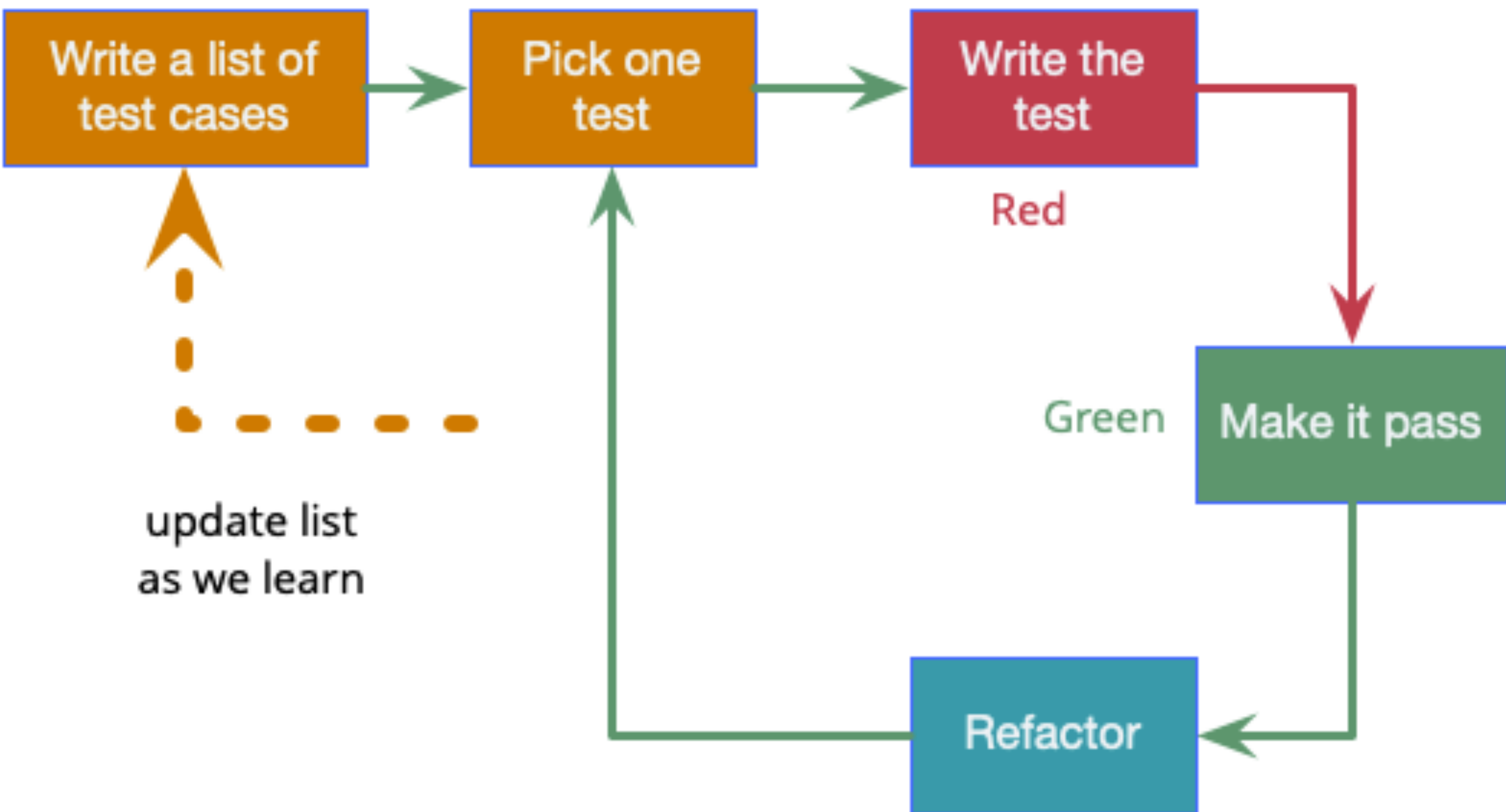
Red

Green

Make it pass

Refactor

update list
as we learn



Liste der Testfälle

Entwicklungszyklen

1. Umrechnung von Celsius in Fahrenheit
2. Temperaturwert abfragen
3. Umrechnung eines abgefragten Wertes von Celsius in Fahrenheit
4. Umrechnung von Fahrenheit in Celsius
5. Umrechnung eines abgefragten Wertes von Fahrenheit in Celsius





#1 Umrechnung von Celsius in Fahrenheit

```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("# Temperature Converter");
9         System.out.println();
10
11        System.out.println("## Test 1) Convert Celsius to Fahrenheit");
12        test(-16.26f, converter.convertCelsiusToFahrenheit(-26.81f));
13        test(32.0f, converter.convertCelsiusToFahrenheit(0.0f));
14        test(531.79f, converter.convertCelsiusToFahrenheit(277.66f));
15    }
16
17    public static void test(float expected, float actual) {
18        // Relatively low accuracy is fine for a consumer converter.
19        if (Math.abs(expected - actual) < 0.01f) {
20            System.out.println("✅ Expected: " + expected + " - Actual: " + actual);
21        } else {
22            System.out.println("❌ Expected: " + expected + " - Actual: " + actual);
23        }
24    }
25
26 }
27
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

Test


```
1 public class TemperatureConverter {  
2  
3     public float convertCelsiusToFahrenheit(float x) {  
4         return 0.0f;  
5     }  
6  
7 }  
8
```

STDIN

Input for the program (Optional)

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✗ Expected: -16.26 - Actual: 0.0

✗ Expected: 32.0 - Actual: 0.0

✗ Expected: 531.79 - Actual: 0.0

Red

```
1 ▾ public class TemperatureConverter {  
2  
3 ▾     public float convertCelsiusToFahrenheit(float x) {  
4         return (x * 9 / 5) + 32;  
5     }  
6  
7 }  
8 |
```

STDIN

Input for the program (Optional)

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✓ Expected: -16.26 - Actual: -16.258

✓ Expected: 32.0 - Actual: 32.0

✓ Expected: 531.79 - Actual: 531.78796

Green

```
1 ▾ public class TemperatureConverter {  
2  
3 ▾     public float convertCelsiusToFahrenheit(float celsius) {  
4         return (celsius * 9 / 5) + 32;  
5     }  
6  
7 }  
8 |
```

STDIN

Input for the program (Optional)

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✓ Expected: -16.26 - Actual: -16.258

✓ Expected: 32.0 - Actual: 32.0

✓ Expected: 531.79 - Actual: 531.78796

Refactor



#2 Temperaturwert abfragen

```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("# Temperature Converter");
9         System.out.println();
10
11        System.out.println("## Test 1) Convert Celsius to Fahrenheit");
12        test(-16.26f, converter.convertCelsiusToFahrenheit(-26.81f));
13        test(32.0f, converter.convertCelsiusToFahrenheit(0.0f));
14        test(531.79f, converter.convertCelsiusToFahrenheit(277.66f));
15
16        System.out.println();
17        System.out.println("## Test 2) Prompt for temperature value");
18        TemperatureConverterHelper helper = new TemperatureConverterHelper();
19        float userValue = helper.promptUserTemperature();
20        test(21.18f, userValue);
21    }
22
23    public static void test(float expected, float actual) {
24        // Relatively low accuracy is fine for a consumer converter.
25        if (Math.abs(expected - actual) < 0.01f) {
26            System.out.println("✅ Expected: " + expected + " - Actual: " + actual);
27        } else {
28            System.out.println("❌ Expected: " + expected + " - Actual: " + actual);
29        }
30    }
31 }
32
33
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

Test

```
1 import java.util.Scanner;
2
3 public class TemperatureConverterHelper {
4
5     public float promptUserTemperature() {
6         return 0.0f;
7     }
8
9 }
10
```

STDIN

Input for the program (Optional)

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✓ Expected: -16.26 - Actual: -16.258

✓ Expected: 32.0 - Actual: 32.0

✓ Expected: 531.79 - Actual: 531.78796

Test 2) Prompt for temperature value

✗ Expected: 21.18 - Actual: 0.0

Red


```
1 import java.util.Scanner;
2
3 public class TemperatureConverterHelper {
4
5     public float promptUserTemperature() {
6         System.out.print("?");
7
8         Scanner scanner = new Scanner(System.in);
9         float input = scanner.nextFloat();
10
11         return input;
12     }
13 }
14 }
15 |
```

STDIN

21.18

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✅ Expected: -16.26 - Actual: -16.258

✅ Expected: 32.0 - Actual: 32.0

✅ Expected: 531.79 - Actual: 531.78796

Test 2) Prompt for temperature value

?✅ Expected: 21.18 - Actual: 21.18

Green

```
1 import java.util.Scanner;
2
3 public class TemperatureConverterHelper {
4
5     public float promptUserTemperature() {
6         System.out.print("Please enter temperature: ");
7
8         Scanner scanner = new Scanner(System.in);
9         float input = scanner.nextFloat();
10        scanner.close();
11
12        System.out.println();
13
14        return input;
15    }
16
17 }
18
```

STDIN

21.18

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✓ Expected: -16.26 - Actual: -16.258

✓ Expected: 32.0 - Actual: 32.0

✓ Expected: 531.79 - Actual: 531.78796

Test 2) Prompt for temperature value

Please enter temperature:

✓ Expected: 21.18 - Actual: 21.18

Refactor



#3 Umrechnung eines abgefragten Wertes von Celsius in Fahrenheit


```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("# Temperature Converter");
9         System.out.println();
10
11        System.out.println("## Test 1) Convert Celsius to Fahrenheit");
12        test(-16.26f, converter.convertCelsiusToFahrenheit(-26.81f));
13        test(32.0f, converter.convertCelsiusToFahrenheit(0.0f));
14        test(531.79f, converter.convertCelsiusToFahrenheit(277.66f));
15
16        System.out.println();
17        System.out.println("## Test 2) Prompt for temperature value");
18        TemperatureConverterHelper helper = new TemperatureConverterHelper();
19        float userValue = helper.promptUserTemperature();
20        test(21.18f, userValue);
21
22        System.out.println();
23        System.out.println("## Test 3) Convert prompted value (" + userValue + ") from Celsius to Fahrenheit");
24        test(70.12f, converter.convertCelsiusToFahrenheit(userValue));
25    }
26
27    public static void test(float expected, float actual) {
28        // Relatively low accuracy is fine for a consumer converter.
29        if (Math.abs(expected - actual) < 0.01f) {
30            System.out.println("✅ Expected: " + expected + " - Actual: " + actual);
31        } else {
32            System.out.println("❌ Expected: " + expected + " - Actual: " + actual);
33        }
34    }
35
36 }
37
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

Test


```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("# Temperature Converter");
9         System.out.println();
10
11        System.out.println("## Test 1) Convert Celsius to Fahrenheit");
12        test(-16.26f, converter.convertCelsiusToFahrenheit(-16.81f));
13        test(32.0f, converter.convertCelsiusToFahrenheit(0.0f));
14        test(531.79f, converter.convertCelsiusToFahrenheit(277.66f));
15
16        System.out.println();
17        System.out.println("## Test 2) Prompt for temperature value");
18        TemperatureConverterHelper helper = new TemperatureConverterHelper();
19        float userValue = helper.promptUserTemperature();
20        test(21.18f, userValue);
21
22        System.out.println();
23        System.out.println("## Test 3) Convert prompted value (" + userValue + ") from Celsius to Fahrenheit");
24        test(70.12f, converter.convertCelsiusToFahrenheit(userValue));
25    }
26
27    public static void test(float expected, float actual) {
28        // Relatively low accuracy is fine for a consumer converter.
29        if (Math.abs(expected - actual) < 0.01f) {
30            System.out.println("✅ Expected: " + expected + " - Actual: " + actual);
31        } else {
32            System.out.println("❌ Expected: " + expected + " - Actual: " + actual);
33        }
34    }
35
36 }
37
```

STDIN

21.18

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✅ Expected: -16.26 - Actual: -16.258

✅ Expected: 32.0 - Actual: 32.0

✅ Expected: 531.79 - Actual: 531.78796

Test 2) Prompt for temperature value

Please enter temperature:

✅ Expected: 21.18 - Actual: 21.18

Test 3) Convert prompted value (21.18) from Celsius to Fahrenheit

✅ Expected: 70.12 - Actual: 70.124

Green



#4 Umrechnung von Fahrenheit in Celsius

```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("# Temperature Converter");
9         System.out.println();
10
11        System.out.println("## Test 1) Convert Celsius to Fahrenheit");
12        test(-16.26f, converter.convertCelsiusToFahrenheit(-26.81f));
13        test(32.0f, converter.convertCelsiusToFahrenheit(0.0f));
14        test(531.79f, converter.convertCelsiusToFahrenheit(277.66f));
15
16        System.out.println();
17        System.out.println("## Test 2) Prompt for temperature value");
18        TemperatureConverterHelper helper = new TemperatureConverterHelper();
19        float userValue = helper.promptUserTemperature();
20        test(21.18f, userValue);
21
22        System.out.println();
23        System.out.println("## Test 3) Convert prompted value (" + userValue + ") from Celsius to Fahrenheit");
24        test(70.12f, converter.convertCelsiusToFahrenheit(userValue));
25
26        System.out.println();
27        System.out.println("## Test 4) Convert Fahrenheit to Celsius");
28        test(-73.25f, converter.convertFahrenheitToCelsius(-99.85f));
29        test(-17.78f, converter.convertFahrenheitToCelsius(0.0f));
30        test(373.62f, converter.convertFahrenheitToCelsius(704.51f));
31    }
32
33    public static void test(float expected, float actual) {
34        // Relatively low accuracy is fine for a consumer converter.
35        if (Math.abs(expected - actual) < 0.01f) {
36            System.out.println("✅ Expected: " + expected + " - Actual: " + actual);
37        } else {
38            System.out.println("❌ Expected: " + expected + " - Actual: " + actual);
39        }
40    }
41 }
42
43
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

Test



```
1 public class TemperatureConverter {  
2  
3     public float convertCelsiusToFahrenheit(float celsius) {  
4         return (celsius * 9 / 5) + 32;  
5     }  
6  
7     public float convertFahrenheitToCelsius(float fahrenheit) {  
8         return 0.0f;  
9     }  
10  
11 }  
12
```

STDIN

21.18

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✓ Expected: -16.26 - Actual: -16.258

✓ Expected: 32.0 - Actual: 32.0

✓ Expected: 531.79 - Actual: 531.78796

Test 2) Prompt for temperature value

Please enter temperature:

✓ Expected: 21.18 - Actual: 21.18

Test 3) Convert prompted value (21.18) from Celsius to Fahrenheit

✓ Expected: 70.12 - Actual: 70.124

Test 4) Convert Fahrenheit to Celsius

✗ Expected: -73.25 - Actual: 0.0

✗ Expected: -17.78 - Actual: 0.0

✗ Expected: 373.62 - Actual: 0.0

Red



```
1 public class TemperatureConverter {
2
3     public float convertCelsiusToFahrenheit(float celsius) {
4         return (celsius * 9 / 5) + 32;
5     }
6
7     public float convertFahrenheitToCelsius(float fahrenheit) {
8         return (fahrenheit - 32) * 5 / 9;
9     }
10
11 }
12
```

STDIN

21.18

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✓ Expected: -16.26 - Actual: -16.258

✓ Expected: 32.0 - Actual: 32.0

✓ Expected: 531.79 - Actual: 531.78796

Test 2) Prompt for temperature value

Please enter temperature:

✓ Expected: 21.18 - Actual: 21.18

Test 3) Convert prompted value (21.18) from Celsius to Fahrenheit

✓ Expected: 70.12 - Actual: 70.124

Test 4) Convert Fahrenheit to Celsius

✓ Expected: -73.25 - Actual: -73.25

✓ Expected: -17.78 - Actual: -17.777779

✓ Expected: 373.62 - Actual: 373.61667

Green



#5 Umrechnung eines abgefragten Wertes von Fahrenheit in Celsius

```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("# Temperature Converter");
9         System.out.println();
10
11        System.out.println("## Test 1) Convert Celsius to Fahrenheit");
12        test(-16.26f, converter.convertCelsiusToFahrenheit(-26.81f));
13        test(32.0f, converter.convertCelsiusToFahrenheit(0.0f));
14        test(531.79f, converter.convertCelsiusToFahrenheit(277.66f));
15
16        System.out.println();
17        System.out.println("## Test 2) Prompt for temperature value");
18        TemperatureConverterHelper helper = new TemperatureConverterHelper();
19        float userValue = helper.promptUserTemperature();
20        test(21.18f, userValue);
21
22        System.out.println();
23        System.out.println("## Test 3) Convert prompted value (" + userValue + ") from Celsius to Fahrenheit");
24        test(70.12f, converter.convertCelsiusToFahrenheit(userValue));
25
26        System.out.println();
27        System.out.println("## Test 4) Convert Fahrenheit to Celsius");
28        test(-73.25f, converter.convertFahrenheitToCelsius(-99.85f));
29        test(-17.78f, converter.convertFahrenheitToCelsius(0.0f));
30        test(373.62f, converter.convertFahrenheitToCelsius(704.51f));
31
32        System.out.println();
33        System.out.println("## Test 5) Convert prompted value (" + userValue + ") from Fahrenheit to Celsius");
34        test(-6.01f, converter.convertFahrenheitToCelsius(userValue));
35    }
36
37    public static void test(float expected, float actual) {
38        // Relatively low accuracy is fine for a consumer converter.
39        if (Math.abs(expected - actual) < 0.01f) {
40            System.out.println("✅ Expected: " + expected + " - Actual: " + actual);
41        } else {
42            System.out.println("❌ Expected: " + expected + " - Actual: " + actual);
43        }
44    }
45 }
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

Test


```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("# Temperature Converter");
9         System.out.println();
10
11        System.out.println("## Test 1) Convert Celsius to Fahrenheit");
12        test(-16.26f, converter.convertCelsiusToFahrenheit(-26.81f));
13        test(32.0f, converter.convertCelsiusToFahrenheit(0.0f));
14        test(531.79f, converter.convertCelsiusToFahrenheit(277.66f));
15
16        System.out.println();
17        System.out.println("## Test 2) Prompt for temperature value");
18        TemperatureConverterHelper helper = new TemperatureConverterHelper();
19        float userValue = helper.promptUserTemperature();
20        test(21.18f, userValue);
21
22        System.out.println();
23        System.out.println("## Test 3) Convert prompted value (" + userValue + ") from Celsius to Fahrenheit");
24        test(70.12f, converter.convertCelsiusToFahrenheit(userValue));
25
26        System.out.println();
27        System.out.println("## Test 4) Convert Fahrenheit to Celsius");
28        test(-73.25f, converter.convertFahrenheitToCelsius(-99.85f));
29        test(-17.78f, converter.convertFahrenheitToCelsius(0.0f));
30        test(373.62f, converter.convertFahrenheitToCelsius(704.51f));
31
32        System.out.println();
33        System.out.println("## Test 5) Convert prompted value (" + userValue + ") from Fahrenheit to Celsius");
34        test(-6.01f, converter.convertFahrenheitToCelsius(userValue));
35    }
36
37    public static void test(float expected, float actual) {
38        // Relatively low accuracy is fine for a consumer converter.
39        if (Math.abs(expected - actual) < 0.01f) {
40            System.out.println("✅ Expected: " + expected + " - Actual: " + actual);
41        } else {
42            System.out.println("❌ Expected: " + expected + " - Actual: " + actual);
43        }
44    }
45 }
```

STDIN

21.18

Output:

Temperature Converter

Test 1) Convert Celsius to Fahrenheit

✅ Expected: -16.26 - Actual: -16.258

✅ Expected: 32.0 - Actual: 32.0

✅ Expected: 531.79 - Actual: 531.78796

Test 2) Prompt for temperature value

Please enter temperature:

✅ Expected: 21.18 - Actual: 21.18

Test 3) Convert prompted value (21.18) from Celsius to Fahrenheit

✅ Expected: 70.12 - Actual: 70.124

Test 4) Convert Fahrenheit to Celsius

✅ Expected: -73.25 - Actual: -73.25

✅ Expected: -17.78 - Actual: -17.777779

✅ Expected: 373.62 - Actual: 373.61667

Test 5) Convert prompted value (21.18) from Fahrenheit to Celsius

✅ Expected: -6.01 - Actual: -6.011111

Testautomati- sierung mit JUnit

Programmierung 2 und 3

TESTING

TEST EXPLORER

Filter (e.g. text, !exclude, @tag)

5/5 631ms

Temperaturumwandlung - JUnit Jupiter 27ms

<Default Package> 27ms

TemperatureConverterTest Temperature Converter: 27ms

convertCelsiusToFahrenheit() Convert Celsius to Fahrenheit: 1.0...

promptForTemperatureValue() Prompt for temperature value: 1...

convertPromptedValueCelsiusToFahrenheit() Convert prompt...

convertFahrenheitToCelsius() Convert Fahrenheit to Celsius: 12...

convertPromptedValueFahrenheitToCelsius() Convert prompt...

TEST COVERAGE

src 100.00%

TemperatureConverter.java 100.00%

TemperatureConverterHelper.java 100.00%

TemperatureConverterTest.java 100.00%

TemperatureConverterTest.java 1, A

TemperatureConverter.java 1, A

TemperatureConverterHelper.java

```
src > J TemperatureConverterTest.java > ...
1  import static org.junit.jupiter.api.Assertions.assertEquals;
2
3  import org.junit.jupiter.api.DisplayName;
4  import org.junit.jupiter.api.Test;
5  import org.junitpioneer.jupiter.StdIo;
6
7  @DisplayName("Temperature Converter")
8  public class TemperatureConverterTest {
9
10
11     @Test
12     @DisplayName("Convert Celsius to Fahrenheit")
13     public void convertCelsiusToFahrenheit() {
14         TemperatureConverter converter = new TemperatureConverter();
15         assertEquals(-16.26f, converter.convertCelsiusToFahrenheit(-26.81f), 0.01f);
16         assertEquals(32.0f, converter.convertCelsiusToFahrenheit(celsius:0.0f), 0.01f);
17         assertEquals(531.79f, converter.convertCelsiusToFahrenheit(celsius:277.66f), 0.01f);
18     }
19
20     @Test
21     @StdIo({ "21.18" })
22     @DisplayName("Prompt for temperature value")
23     public void promptForTemperatureValue() {
24         TemperatureConverterHelper helper = new TemperatureConverterHelper();
25         float userValue = helper.promptUserTemperature();
26         assertEquals(21.18f, userValue, 0.01f);
27     }
28
29     @Test
30     @StdIo({ "21.18" })
31     @DisplayName("Convert prompted value (21.18) from Celsius to Fahrenheit")
32     public void convertPromptedValueCelsiusToFahrenheit() {
33         TemperatureConverter converter = new TemperatureConverter();
34         TemperatureConverterHelper helper = new TemperatureConverterHelper();
35         float userValue = helper.promptUserTemperature();
```

PROBLEMS 10

OUTPUT

DEBUG CONSOLE

TEST RESULTS

TERMINAL

PORTS

SPELL CHECKER 5

Filter (e.g. text, !ex...

Please enter temperature:

Please enter temperature:

Please enter temperature:

Ln 58, Col 1

Spaces: 2

UTF-8

LF

{ } Java

Prettier

 Welcher **eine** Gedanke zu
„ Entwicklungszyklen“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Pair Programming

🎓 Vorgehen und Testcode

🤝 Pair Programming

🎯 Entwickeln Sie einen Konverter, der zwischen Kilometern pro Stunde und Meilen pro Stunde umrechnet!

- ✅ Es muss eine Methode für die Umrechnung „km/h → mph“...
- ✅ und eine für die umgekehrte Richtung geben.



Vorgehen und Testcode

Pair Programming

 **Entwickeln Sie einen Konverter, der zwischen Kilometern pro Stunde und Meilen pro Stunde umrechnet!**

 Es muss eine Methode für die Umrechnung „km/h → mph“...

 und eine für die umgekehrte Richtung geben.

 Entwickeln Sie testgetrieben, verwenden Sie also TDD.

 Am Ende sollten Sie eine Testklasse, wie z. B. `SpeedConverterTestDrive`, ...

 ... und die implementierende Klasse, wie `SpeedConverter`, haben.

```
1 import java.util.*;
2
3 public class SpeedConverterTestDrive {
4     public static void main(String[] args) {
5         SpeedConverter converter = new SpeedConverter();
6
7         System.out.println("# Speed Converter");
8         System.out.println();
9
10        //
11        // TDD cycle #1 / step #1 (pick test)
12        //
13        System.out.println("## Test 1) Convert km/h to mph");
14        test(0.0f, converter.convertKmhToMph(0.0f));
15        test(2.31f, converter.convertKmhToMph(3.71f));
16        test(110.61f, converter.convertKmhToMph(178.01f));
17
18        //
19        // TDD cycle #2 / step #1 (pick test)
20        //
21        System.out.println();
22        System.out.println("## Test 2) Convert mph to km/h");
23        test(0.0f, converter.convertMphToKmh(0.0f));
24        test(31.66f, converter.convertMphToKmh(19.67f));
25        test(145.82f, converter.convertMphToKmh(90.61f));
26    }
27
28    public static void test(float expected, float actual) {
29        // Relatively low accuracy is fine for a consumer converter.
30        if (Math.abs(expected - actual) < 0.01f) {
31            System.out.println("✅ Expected: " + expected + " - Actual: " + actual);
32        } else {
33            System.out.println("❌ Expected: " + expected + " - Actual: " + actual);
34        }
35    }
36 }
37
```

STDIN

Input for the program (Optional)

Output:

Speed Converter

Test 1) Convert km/h to mph

✅ Expected: 0.0 - Actual: 0.0

✅ Expected: 2.31 - Actual: 2.3052871

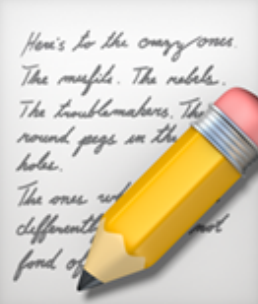
✅ Expected: 110.61 - Actual: 110.61028

Test 2) Convert mph to km/h

✅ Expected: 0.0 - Actual: 0.0

✅ Expected: 31.66 - Actual: 31.655796

✅ Expected: 145.82 - Actual: 145.82266

 Welcher **eine** Gedanke zu
„ Vorgehen und Testcode“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Schleifen

for-each-Schleife

↻ Schleifen

Wiederhole für jedes
→ (for each) Element
im String-Array „names“
diesen Code!

```
for (String name: names) {...}
```

Weise dabei das
jeweilige Element
der String-Variablen
„name“ zu...

... so dass das jeweilige
Element (der jeweilige
Name) im Code
verwendet werden
kann!

Postinkrement-Operator

 Schleifen

Addiere 1
zum Wert in
der Variablen
hinzu!

numberOfPASSES++

Das Gleiche wie:

numberOfPASSES = numberOfPASSES + 1

break-Anweisung

 Schleifen

Beende die Schleife ohne weitere
Beachtung ihrer Testbedingung!

break;

„Gehen Sie direkt in das Gefängnis, gehen Sie nicht
über Los, ziehen Sie keine 4000 Geldeinheiten ein.“

```
1 import java.util.*;
2
3 public class LoopingDemo {
4     public static void main(String[] args) {
5         Random generator = new Random();
6
7         // initialize array with 10 random numbers
8         int[] numbers = new int[10];
9         for (int i = 0; i < 10; i++) {
10             numbers[i] = generator.nextInt(10); // [0 - 9]
11         }
12
13         System.out.println("Generated array with 10 random numbers:");
14         System.out.println(Arrays.toString(numbers));
15
16         int loopings = 1;
17
18         for (int number: numbers) { // enhanced for loop aka for-each loop
19
20             System.out.println("#" + loopings + " -> " + number);
21
22             // When the value 4 is found...
23             if (number == 4) {
24                 System.out.println("Found number 4 at index: " + (loopings - 1));
25                 // ... the break statement causes the for loop to exit:
26                 break;
27             }
28
29             loopings++; // loopings = loopings + 1
30         }
31
32     }
33
34 }
35
```

STDIN

Input for the program (Optional)

Output:

Generated array with 10 random numbers:

[8, 7, 5, 1, 2, 6, 4, 7, 2, 4]

#1 -> 8

#2 -> 7

#3 -> 5

#4 -> 1

#5 -> 2

#6 -> 6

#7 -> 4

Found number 4 at index: 6

```
1 import java.util.*;
2
3 public class IncrementOperatorsDemo {
4
5     public static void main(String[] args) {
6         System.out.println("# Increment Operators");
7         System.out.println();
8
9         System.out.println("## Pre-Increment Operator");
10
11         int x = 0;
12         System.out.println("x = 0 --> x: " + x);
13
14         int y = ++x; // Increment x. Then assign value in x to y.
15         System.out.println("y = ++x --> y: " + y + ", x: " + x);
16
17         System.out.println();
18         System.out.println("-> Post-Increment Operator");
19
20         int a = 0;
21         System.out.println("a = 0 --> a: " + a);
22
23         int b = a++; // Assign value in a to b. Then increment a.
24         System.out.println("b = a++ --> b: " + b + ", a: " + a);
25
26         System.out.println();
27         System.out.println("## While loop counts from 1 to 10 with pre-increment operator");
28         int c = 0;
29         while (++c <= 10) {
30             System.out.println(c);
31         }
32
33         System.out.println();
34         System.out.println("## Same while loop with post-increment operator");
35         int d = 0;
36         while (d++ <= 10) { // Doh!
37             System.out.println(d);
38         }
39
40     }
41
42 }
43
```

STDIN

Input for the program (Optional)

Output:

Increment Operators

Pre-Increment Operator

x = 0 --> x: 0

y = ++x --> y: 1, x: 1

-> Post-Increment Operator

a = 0 --> a: 0

b = a++ --> b: 0, a: 1

While loop counts from 1 to 10 with pre-increment operator

1

2

3

4

5

6

7

8

9

10

Same while loop with post-increment operator

1

2

3

4

5

6

7

8

9

10

11

! ? Quiz

↻ Schleifen

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Die „reguläre“ for-Schleife vereint die drei Teile Initialisierung, boolescher Test und Wiederholungsausdruck im Schleifenkopf.

Richtig!

Die `while`-Schleife enthält typischerweise im Schleifenkopf nur den booleschen Test. Die Initialisierung sollte außerhalb der Schleife erfolgen. Der Wiederholungsausdruck erfolgt innerhalb der Schleife.

Richtig!

for- und while-Schleifen können immer ineinander umgewandelt werden, da beide die Grundprinzipien der Schleifenkontrolle implementieren.

Richtig!

Bevorzugen Sie for-Schleifen gegenüber while-Schleifen, wenn Sie wissen, wie oft der Code in der Schleife ausgeführt wird.

Richtig!

Die erweiterte (enhanced) for-Schleife ist eine aufwändigere Art, über ein Array zu iterieren.

Die erweiterte for-Schleife kann dies vereinfachen.

Der Inkrement-Operator wird verwendet, um zu einem `int` eins hinzuzufügen (z. B. `counter ++`),
der Dekrement-Operator wird verwendet, um eins abzuziehen (z. B. `countdown --`).

Richtig!

Es spielt keine Rolle, ob wir den Prä- oder Postinkrement-Operator verwenden. Das Ergebnis ist am Ende immer dasselbe.

Zum Beispiel in Zuweisungen oder Schleifen kann dies einen großen Unterschied machen.

Es gibt keine Möglichkeit, eine `while`- oder `for`-Schleife vorzeitig zu beenden. Die Schleife wird immer durchlaufen, solange die Testbedingung wahr ist.

Mit Hilfe der `break`-Anweisung ist es möglich, eine Schleife vorzeitig zu verlassen.

 Welcher **eine** Gedanke zu
„ Schleifen“ war für Sie am
wichtigsten?  Schreiben Sie
ihn auf!

 Durchatmen und  Sortieren



Elementare Typen casten

Typumwandlung durch „(Cast)“

 Elementare Typen casten

int-Variable, die eine
Zufallszahl enthalten soll.

Typumwandlung
(Cast)

```
int rand = (int) (Math.random() * 5)
```

 Der int-Cast „schneidet“
den Nachkommateil des
double-Wertes ab!



 Double
zwischen
0 und 4,999...

```
1 import java.util.*;
2
3 public class CastingDemo {
4
5     public static void main(String[] args) {
6
7         // Double value with a positive sign, greater than
8         // or equal to 0.0 and less than 1.0.
9         double randomValue = 0.376504640692349; // Math.random();
10
11         System.out.println("-> Untreated randomValue");
12         System.out.println("randomValue          : " + randomValue);
13         System.out.println("randomValue * 5      : " + (randomValue * 5));
14
15         System.out.println();
16         System.out.println("-> Casted randomValue");
17         System.out.println("(int) randomValue      : " + (int) randomValue);
18         System.out.println("(int) randomValue * 5  : " + (int) randomValue * 5); // Doh!
19         System.out.println("(int) (randomValue * 5) : " + (int) (randomValue * 5)); // Phew!
20
21         System.out.println();
22         // Math.round() returns the closest int to the argument.
23         System.out.println("-> Using Math.round() instead of cast");
24         System.out.println("Math.round(randomValue) : " + Math.round(randomValue));
25         System.out.println("Math.round(randomValue * 5): " + Math.round(randomValue * 5));
26
27     }
28
29 }
30
```

STDIN

Input for the program (Optional)

Output:

-> Untreated randomValue

randomValue	:	0.376504640692349
randomValue * 5	:	1.8825232034617452

-> Casted randomValue

(int) randomValue	:	0
(int) randomValue * 5	:	0
(int) (randomValue * 5)	:	1

-> Using Math.round() instead of cast

Math.round(randomValue)	:	0
Math.round(randomValue * 5)	:	2

! ? Quiz

👖 Elementare Typen casten

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



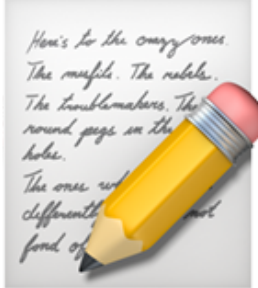

```
long a = 40002; short b = (short) a;
```

b hat jetzt den um die ersten Ziffern
„gestutzten“ Wert 2.

b hat den Wert -25534. Die Bits in der Bitdarstellung werden abgeschnitten.

Für eine korrekte Rundung kann die Methode `Math.round()` verwendet werden.

Richtig!

 Welcher **eine** Gedanke zu
„ Elementare Typen casten“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Ein Programm schreiben

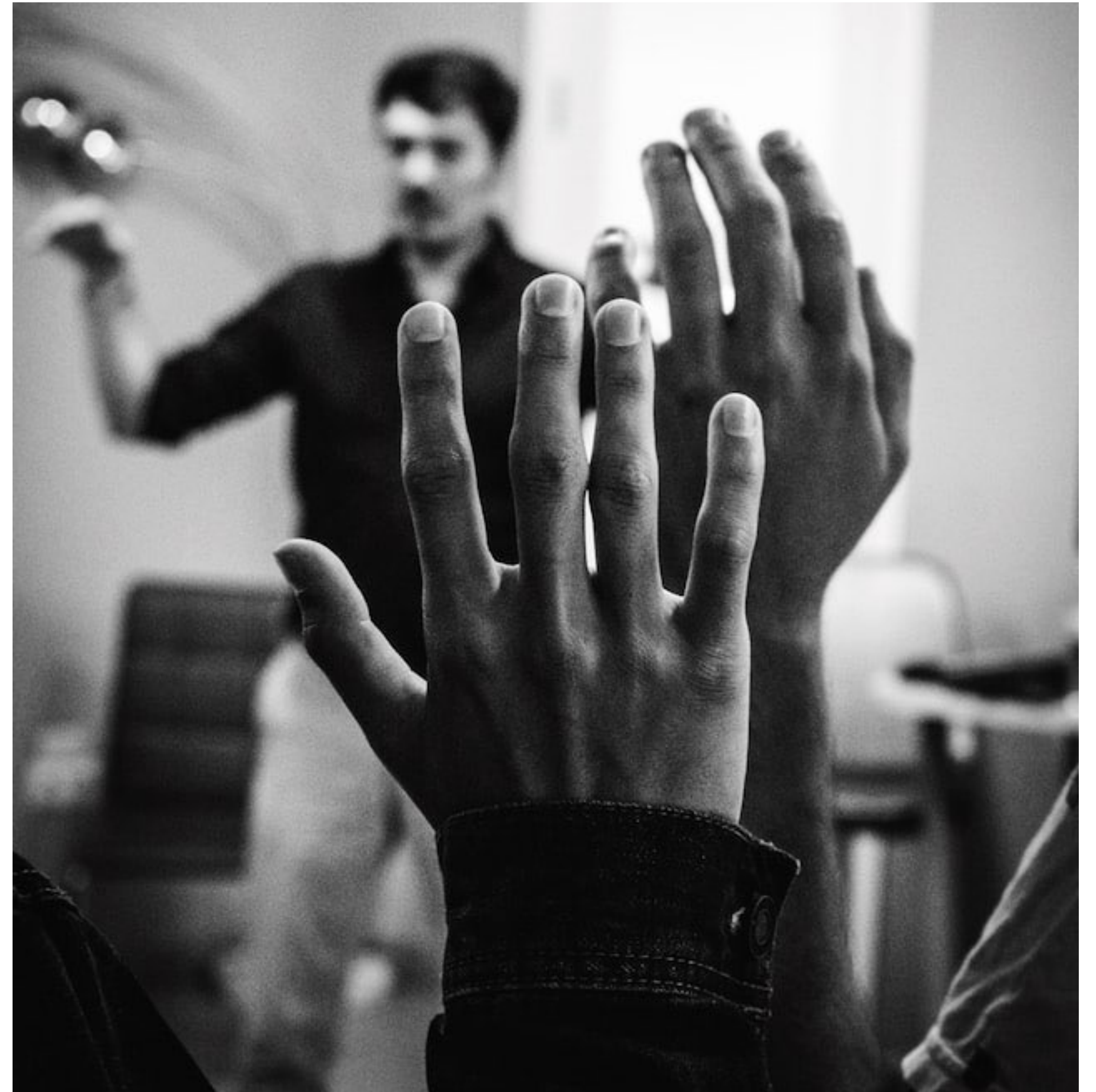
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Ein Programm schreiben?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Ein Programm schreiben



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

